

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a309bc1de7ed1bbe1.jsonl`  
- SHA-256: `f2f15fc097cd29884127ef4175626a157942fab4eee6cf2036a06e5db2990b69`  
- Source modified: `2026-05-31T04:19:11+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:18:20.860Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

****URL:**** <https://github.com/VikParuchuri/surya/blob/master/LICENSE>

****Title:**** Surya OCR LICENSE ? GPL-3.0 code + OpenRAIL-M weights (revenue-gated)

****Found via:**** Model-weights license minefield (commercial-safety, decisive) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:18:23.223Z)

I'll fetch the Surya OCR LICENSE page to extract the relevant claims about its licensing.

3. assistant (2026-05-31T04:18:29.551Z)

I'll fetch the Surya OCR LICENSE page now.

4. user (2026-05-31T04:18:37.049Z)

License Analysis for Surya Repository

License Governing the Code

The ****Apache License 2.0**** governs the code in the Surya repository. This is a permissive open-source license.

License Governing Model Weights

The provided content does not include information about model weights or their licensing terms. Only the code license is presented in this document.

Commercial Use Restrictions

The Apache 2.0 license contains **no revenue restrictions, commercial use caps, or funding thresholds**. The license explicitly grants:

"perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license" (Section 2)

Commercial use is permitted without limitation or special conditions.

Other License Types

The document does **not mention**:

- GPL
- OpenRAIL-M
- CC-BY-NC
- Dual-licensing arrangements

Key Takeaway

Apache 2.0 is a permissive license allowing commercial use, modification, and distribution without revenue restrictions, provided that users include license notices and attribution statements.

5. assistant (2026-05-31T04:18:41.766Z)

The WebFetch result shows the LICENSE file at HEAD is now Apache 2.0, which contradicts the title's claim of "GPL-3.0 code + OpenRAIL-M weights (revenue-gated)". This is a significant discrepancy. The title appears to reflect an older state of the repo. Surya's licensing is known to have changed over time. Let me verify the current state more carefully and check the repo's README and any commercial-licensing notes, since the LICENSE file at `master` may have been relicensed.

Let me check the repository README and the git history of the LICENSE file to understand the discrepancy.

6. user (2026-05-31T04:18:46.820Z)

License Identification

The exact license header at the top reads:

```
"Apache License
Version 2.0, January 2004
http://www.apache.org/licenses/"
```

This is **Apache License 2.0** (not GPL-3.0 or OpenRAIL-M).

First 20 Lines (Verbatim)

```
"Apache License
Version 2.0, January 2004
http://www.apache.org/licenses/
```

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is

granting the License."

Special Clauses

This license contains **no** clauses about:

- Model weights
- Revenue thresholds
- Commercial use restrictions

The Apache 2.0 license is permissive, explicitly allowing commercial redistribution. It does include a patent grant provision and requires attribution notices to be preserved, but has no revenue-based limitations or restrictions on commercial applications.

7. user (2026-05-31T04:18:49.203Z)

Surya Licensing Summary

Code License: Apache 2.0

- "The Surya code is licensed under Apache 2.0"

Model Weights License: Modified OpenRAIL-M

- "The model weights use a modified AI Pubs Open Rail-M license (free for research, personal use, and startups under \$5M funding/revenue)"

Commercial Use Gate: \$5M threshold

- The modified OpenRAIL-M license permits free use for startups with funding or revenue below \$5 million
- Broader commercial licensing of model weights requires visiting their pricing page

Key Distinction: The code and model weights are governed by separate licenses, with the weights having a revenue/funding cap that triggers the need for commercial licensing agreements.

8. assistant (2026-05-31T04:18:57.774Z)

This clarifies the situation. The LICENSE file at `master` is now Apache 2.0 (the code license), but the README confirms the model weights use a modified OpenRAIL-M license with a \$5M funding/revenue threshold. The source title's framing of "GPL-3.0 code" is now stale ? the code was relicensed from GPL-3.0 to Apache 2.0 at some point. However, the README is the authoritative source for the current weights license, which is the decisive fact for commercial safety.

Let me note that the specific URL I was asked to fetch (the LICENSE file) now contains Apache 2.0, not GPL-3.0. The README is where the weights-license restriction lives. Both fetches succeeded and corroborate each other. I have enough to extract falsifiable claims.

Key findings for the research question:

- **Code license at master:** Apache 2.0 (the title's "GPL-3.0" claim is outdated ? code was relicensed)
- **Weights license:** modified AI Pubs OpenRAIL-M, free only for research/personal use and orgs under \$5M funding/revenue ? i.e., **revenue-gated**, NOT a permissive license, fails muneem's MIT/BSD/Apache requirement for weights
- This corroborates the memory note and research-question warning that "Surya and Marker use GPL-or-commercial dual licensing" ? the practical effect (revenue-gated weights) holds even though the code license is now Apache 2.0

This is a primary source (the official project repository ? the canonical authority on its own license).

9. user (2026-05-31T04:19:04.900Z)

Structured output provided successfully

10. assistant (2026-05-31T04:19:11.568Z)

Done. The fetch succeeded and revealed an important nuance that corrects the source's title.

Key findings:

- The **LICENSE** file at `master` is now Apache 2.0 (code), not GPL-3.0 as the title states ? Surya's code was relicensed at some point. The "GPL-3.0 code" framing is stale for the current branch.
- The **decisive commercial-safety fact still holds**: the README confirms Surya's **model weights** use a modified AI Pubs OpenRAIL-M license, free only for research/personal use and orgs under **\$5M funding/revenue**, with a paid commercial license required above that. This is **revenue-gated** and fails muneem's MIT/BSD/Apache requirement for weights.
- This corroborates muneem's existing memory note (commercial-license-safety) and the research-question warning that Surya uses "GPL-or-commercial dual licensing" ? the practical effect (restricted, revenue-gated weights) is real regardless of the code license change.

Source quality: **primary** (the project's own canonical license/README). One important caveat flagged in the claims: the title's "GPL-3.0 code" is no longer accurate for `master`; the restriction now lives entirely in the **weights** license, which is why verifying the weights license separately (not just the repo LICENSE file) is essential.